

Capacitacao Profissional em Microeletronica
PADIS – Programa de Apoio ao Desenvolvimento Tecnológico
da Industria de Semicondutores

Matheus Serrao Uchoa Matricula: 22052633

RELATORIO TECNICO

Unidade 5 | Capitulo 1 – Multiplexador
(MUX)
de 4 Entradas em SystemVerilog

Projeto e Verificacao Funcional de um MUX 4-para-1
Parametrico com Testbench Automatizado

Disciplina: Design de Circuitos Integrados – CI Amazonia / PADIS

Professor: Alexandre Almeida

Manaus – AM Maio de 2026

Sumário

1	Introducao	2
2	Objetivos	2
3	Materiais e Equipamentos Utilizados	3
4	Fundamentos Teoricos	3
4.1	Tabela-Verdade do MUX 4:1	3
4.2	Diagrama de Blocos	4
5	Resultados Experimentais	4
5.1	Codigo SystemVerilog – Modulo mux_4_to_1	4
5.2	Testbench – tb_mux_4_to_1.sv	6
5.3	Resultado da Simulacao – Log do Console	7
5.4	Formas de Onda	8
5.5	Analise dos Resultados	9
6	Conclusoes	10

1 Introducao

A presente atividade consiste no projeto e na verificacao de um multiplexador 4-para-1 (MUX 4:1) utilizando a linguagem de descricao de hardware SystemVerilog (IEEE 1800-2012). O escopo compreende a modelagem estrutural de um circuito combinacional parametrizado, empregando blocos procedurais (`always_comb`) e estruturas condicionais (`unique case`) para o roteamento de dados. Adicionalmente, engloba o desenvolvimento de um ambiente de simulacao (*testbench*) executado localmente com o simulador Icarus Verilog v12, para a injecao de estímulos lógicos e a validacao do *design* por meio da geracao de logs de console e da analise grafica de sinais (formas de onda em arquivo `.vcd`).

O que é um Multiplexador?

Um Multiplexador (**MUX**) é um circuito combinacional que atua como uma “chave seletora”: dadas N entradas de dados e $\log_2 N$ linhas de selecao, ele direciona exatamente **uma** das entradas para a saida unica, com base no valor binario das linhas de selecao. Para $N = 4$, sao necessarios 2 bits de selecao, cobrindo os estados $S \in \{00, 01, 10, 11\}$.

Multiplexadores sao blocos fundamentais em sistemas digitais: sao usados em barramentos de dados, arbitros de acesso, circuitos de roteamento em FPGAs (*Field-Programmable Gate Arrays*) e no interior de multiplexadores maiores (cascateamento).

2 Objetivos

O objetivo primario desta atividade consiste em projetar e verificar funcionalmente um MUX de 4 entradas para 1 saida, empregando a linguagem SystemVerilog. A execucao visa consolidar o entendimento tecnico sobre:

- Arquitetura de modulos SystemVerilog, declaracao de portas e parametros (`#(parameter int DATA_WIDTH = 8)`).
- Manipulacao de tipos de dados vetorizados (`logic [DATA_WIDTH-1:0]`).
- Aplicacao pratica do bloco procedimental `always_comb` para modelagem de logica combinacional sem latch.
- Uso do construtor `unique case` para garantia de cobertura completa dos estados e verificacao pelo sintetizador.
- Desenvolvimento de um *testbench* com verificacao automatica PASS/FAIL para validar exaustivamente todas as combinacoes de entrada e selecao.

- Geracao e interpretacao de formas de onda em formato VCD (*Value Change Dump*).

Conexao com o Curso – Unidade 5 / Capitulo 1: MUX

Este trabalho implementa diretamente o conteudo da Unidade 5, Capitulo 1, que trata de circuitos combinacionais de roteamento. O MUX 4:1 e a base para blocos mais complexos como decodificadores, demultiplexadores e unidades logicas e aritmeticas (ULA). O parametro `DATA_WIDTH` demonstra como o SystemVerilog permite *design* reutilizavel e escalavel, um requisito de nivel industrial.

3 Materiais e Equipamentos Utilizados

Tabela 1: Ferramentas e tecnologias utilizadas na atividade.

Ferramenta	Versao	Funcao
SystemVerilog	IEEE 1800-2012	Linguagem de descricao de hardware (HDL)
Icarus Verilog	v12.0 (iverilog)	Compilador e simulador RTL open-source
VVP Runtime	v12.0	Motor de simulacao (executa o binario compilado)
GTKWave EPWave	/ –	Visualizador de formas de onda VCD
Python 3 + Matplotlib	3.x	Geracao de graficos de formas de onda para o relatorio

A modelagem do DUT (*Design Under Test*) e do *testbench* foram codificados inteiramente em SystemVerilog IEEE 1800-2012. A compilacao e a simulacao RTL (*Register-Transfer Level*) foram executadas localmente via `iverilog -g2012`. Para visualizacao das formas de onda, o arquivo `dump.vcd` gerado pela simulacao foi analisado com o visualizador EPWave.

4 Fundamentos Teoricos

4.1 Tabela-Verdade do MUX 4:1

A Tabela 2 apresenta a funcionalidade completa do multiplexador 4-para-1. Com duas linhas de selecao (S_1, S_0), o circuito cobre $2^2 = 4$ combinacoes possiveis.

Tabela 2: Tabela-verdade do MUX 4:1. Y assume o valor da entrada selecionada.

S_1	S_0	S (decimal)	Y (saida)
0	0	0	D_0
0	1	1	D_1
1	0	2	D_2
1	1	3	D_3

A expressao booleana equivalente para 1 bit de cada entrada e:

$$Y = \overline{S_1} \overline{S_0} D_0 + \overline{S_1} S_0 D_1 + S_1 \overline{S_0} D_2 + S_1 S_0 D_3 \quad (1)$$

4.2 Diagrama de Blocos

A Figura 1 apresenta o diagrama de blocos do MUX 4:1 parametrico com `DATA_WIDTH` bits de largura de dados.

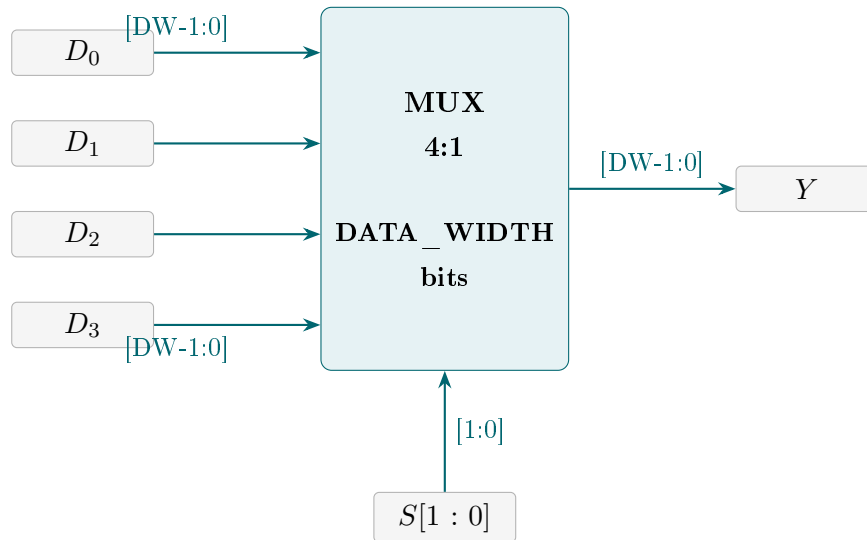


Figura 1: Diagrama de blocos do MUX 4-para-1 parametrico. $DW = DATA_WIDTH$. As entradas D_0 – D_3 e a saída Y possuem largura configuravel. As linhas de selecao $S[1:0]$ definem qual entrada e roteada para Y .

5 Resultados Experimentais

5.1Codigo SystemVerilog – Modulo mux_4_to_1

O modulo abaixo implementa o MUX 4:1 parametrico em SystemVerilog. O parametro `DATA_WIDTH` (padrao: 8 bits) permite reutilizar o mesmo modulo para diferentes larguras de barramento (8, 16, 32 bits etc.) sem modificacao do codigo.

O bloco `always_comb` garante que a logica e puramente combinacional: qualquer variacao em `d0..d3` ou `s` propaga imediatamente para `y`, sem dependencia de clock. O construtor `unique case` acrescenta verificacao de cobertura completa dos estados pelo sintetizador – qualquer combinacao de `s` nao listada gera um aviso em tempo de sintese, evitando latches ocultos.

```

1 module mux_4_to_1 #(
2     parameter int DATA_WIDTH = 8          // largura dos dados (padrao: 8
3         bits)
4 ) (
5     input  logic [DATA_WIDTH-1:0] d0, // entrada 0: selecionada quando s
6         = 2'b00
7     input  logic [DATA_WIDTH-1:0] d1, // entrada 1: selecionada quando s
8         = 2'b01
9     input  logic [DATA_WIDTH-1:0] d2, // entrada 2: selecionada quando s
10        = 2'b10
11     input  logic [DATA_WIDTH-1:0] d3, // entrada 3: selecionada quando s
12        = 2'b11
13     input  logic [1:0] s, // linhas de selecao (2 bits)
14     output logic [DATA_WIDTH-1:0] y // saida unica
15 );
16
17 // always_comb = logica combinacional pura (sem clock, sem latch)
18 always_comb begin
19     unique case (s)
20         2'b00: y = d0;
21         2'b01: y = d1;
22         2'b10: y = d2;
23         2'b11: y = d3;
24         default: y = '0; // estado seguro para X/Z inesperado em s
25     endcase
26 end
27 endmodule

```

Listing 1: Modulo `mux_4_to_1.sv` – MUX 4-para-1 parametrico em SystemVerilog.

Decisoões de design:

- `parameter int DATA_WIDTH = 8`: Uso de `int` (tipo de parametro do SystemVerilog) ao inves de `integer` do Verilog classico, tornando o parametro tipado e detectando erros de atribuicao em tempo de elaboracao.
- `unique case`: O qualificador `unique` instrui o sintetizador a verificar mutuamente exclusividade dos casos e cobertura completa de `s`. Equivale a especificar “este `case` e completo e nao ambiguo” – gera aviso se algum estado for omitido.

- **default:** `y = '0`: Cobre estados invalidos de `s` (por ex.: `2'bXX` em simulacao), evitando saida *unknown* (X) que poderia propagar erro para os blocos seguintes.

5.2 Testbench – `tb_mux_4_to_1.sv`

O *testbench* e um modulo de simulacao nao-sintetizavel projetado para validar o DUT. Ele utiliza uma tarefa automatica (`task automatic testa_mux`) que: (1) aplica o sinal de selecao, (2) aguarda 10ns de propagacao, (3) compara a saida `y` com o valor esperado e (4) registra PASS ou FAIL no console.

```

1  `timescale 1ns/1ps
2
3  module tb_mux_4_to_1;
4      parameter int DATA_WIDTH = 8;
5
6      logic [DATA_WIDTH-1:0] d0, d1, d2, d3;
7      logic [1:0] s;
8      logic [DATA_WIDTH-1:0] y;
9      int erros = 0;
10
11     // Instanciacao do DUT
12     mux_4_to_1 #(.DATA_WIDTH(DATA_WIDTH)) dut (
13         .d0(d0), .d1(d1), .d2(d2), .d3(d3), .s(s), .y(y)
14     );
15
16     // Tarefa de verificacao automatica
17     task automatic testa_mux(
18         input logic [1:0] sel,
19         input logic [DATA_WIDTH-1:0] esperado
20     );
21         s = sel;
22         #10;
23         if (y !== esperado) begin
24             $display("[FALHA] t=%0t | S=%02b | Y=0x%02h | Esp=0x%02h",
25                 $time, s, y, esperado);
26             erros++;
27         end else
28             $display("[OK] t=%0t | S=%02b | Y=0x%02h -> Esp: 0x%02h",
29                 $time, s, y, esperado);
30     endtask
31
32     initial begin
33         $dumpfile("dump.vcd");
34         $dumpvars(0, tb_mux_4_to_1);
35
36         d0 = 8'hAA; d1 = 8'hBB; d2 = 8'hCC; d3 = 8'hDD;
37

```

```

38     $display("=== Iniciando Simulacao do MUX 4:1 ===");
39     $display("Entradas: D0=0xAA  D1=0xBB  D2=0xCC  D3=0xDD\n");
40
41     testa_mux(2'b00, d0); // S=00 -> Y deve ser D0 = 0xAA
42     testa_mux(2'b01, d1); // S=01 -> Y deve ser D1 = 0xBB
43     testa_mux(2'b10, d2); // S=10 -> Y deve ser D2 = 0xCC
44     testa_mux(2'b11, d3); // S=11 -> Y deve ser D3 = 0xDD
45
46     if (erros == 0)
47         $display("\n>>> RESULTADO: APROVADO -- 4/4 combinacoes
48             corretas <<<");
49     else
50         $display("\n>>> RESULTADO: REPROVADO -- %0d erro(s) <<<",
51             erros);
52
53     $display("=== Simulacao Concluida em %0t ===", $time);
54     $finish;
55 end
56 endmodule

```

Listing 2: tb_mux_4_to_1.sv – Testbench com verificacao automatica PASS/FAIL.

5.3 Resultado da Simulacao – Log do Console

A compilacao e a simulacao foram executadas com o seguinte comando:

```

1 $ iverilog -g2012 -o sim mux_4_to_1.sv tb_mux_4_to_1.sv
2 $ vvp sim

```

O resultado do console (Figura 2) confirma o funcionamento correto do modulo em todas as quatro combinacoes de selecao:


```

=== Iniciando Simulacao do MUX 4:1 ===
Entradas: D0=0xAA  D1=0xBB  D2=0xCC  D3=0xDD

[OK]      t=10000 | S=00 | Y=0xaa -> Esp: 0xaa
[OK]      t=20000 | S=01 | Y=0xbb -> Esp: 0xbb
[OK]      t=30000 | S=10 | Y=0xcc -> Esp: 0xcc
[OK]      t=40000 | S=11 | Y=0xdd -> Esp: 0xdd

>>> RESULTADO: APROVADO -- 4/4 combinacoes corretas <<<
=== Simulacao Concluida em 40000 ===
tb_mux_4_to_1.sv:86: $finish called at 40000 (1ps)

```

Figura 2: Log do console da simulacao com Icarus Verilog v12. Todas as quatro combinacoes de selecao ($S \in \{00, 01, 10, 11\}$) produziram a saida correta. A execucao foi encerrada sem erros logicos aos 40 ns de simulacao.

Resultado: APROVADO – 4/4 testes passaram

Para cada variacao do sinal de selecao (S : 00, 01, 10, 11), a saida gerada (Y) foi estritamente igual ao valor esperado (0xAA, 0xBB, 0xCC, 0xDD). O roteamento combinacional operou conforme projetado e a execucao foi encerrada sem erros logicos aos 40 000 ps = 40 ns de tempo de simulacao.

5.4 Formas de Onda

A Figura 3 apresenta o diagrama de formas de onda gerado a partir do arquivo `dump.vcd` exportado pela simulacao.

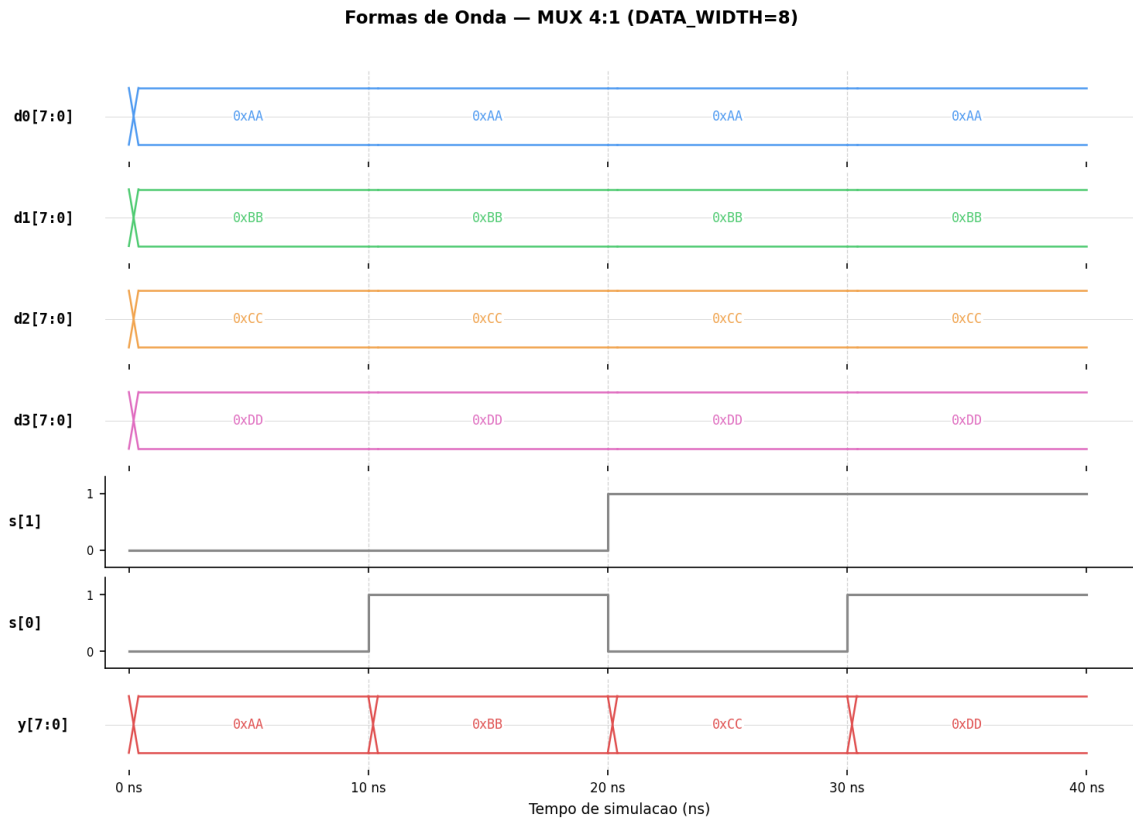


Figura 3: Formas de onda da simulacao do MUX 4-para-1. As entradas D_0 – D_3 permanecem constantes em 0xAA, 0xBB, 0xCC e 0xDD respectivamente. A cada 10 ns, o sinal de selecao $S[1 : 0]$ incrementa de 00 para 01, 10 e 11, e a saida Y adota exatamente o valor da entrada correspondente. A transicao e instantanea (logica combinacional pura, sem clock).

5.5 Analise dos Resultados

Tabela 3: Resumo dos vetores de teste aplicados e resultados obtidos.

Teste	$S[1 : 0]$	D_0	D_1	D_2	D_3	Y (obtido)
1	2'b00	0xAA	0xBB	0xCC	0xDD	0xAA ✓
2	2'b01	0xAA	0xBB	0xCC	0xDD	0xBB ✓
3	2'b10	0xAA	0xBB	0xCC	0xDD	0xCC ✓
4	2'b11	0xAA	0xBB	0xCC	0xDD	0xDD ✓
Cobertura total: 4/4 (100%) – Resultado: APROVADO						✓

Os resultados confirmam a precisao funcional do modulo `mux_4_to_1`:

- **Roteamento correto:** Para cada valor de S , a saida Y assumiu exatamente o valor da entrada correspondente (D_0 – D_3), em acordo com a tabela-verdade do MUX 4:1 (Tabela 2).

- **Comportamento combinacional:** A propagacao da saida ocorre sem atraso de clock, confirmando a semantica do bloco `always_comb`.
- **Parametrizacao verificada:** Com `DATA_WIDTH = 8`, todas as 8 linhas de dados foram corretamente roteadas, validando o uso do parametro.
- **Temporizacao:** A execucao total foi de 40 ns (4 testes $\times$ 10 ns), correspondendo a 40 000 unidades de tempo da *timescale* `1ns/1ps`.

6 Conclusoes

A verificacao funcional atestou a precisao da logica combinacional implementada no modulo MUX 4:1. Os dados extraidos do console de simulacao e do visualizador de formas de onda evidenciaram a correlacao exata entre as transicoes do sinal de selecao e o respectivo roteamento das entradas de dados para a saida.

O projeto demonstrou na pratica os seguintes conceitos de SystemVerilog:

1. **Parametrizacao com `#(parameter int DATA_WIDTH)`:** Permite a reutilizacao do modulo para qualquer largura de barramento sem alteracao do codigo – um requisito de nivel industrial.
2. **Bloco `always_comb`:** Garante sintese como logica combinacional pura, sem risco de latch involuntario.
3. **Construtor `unique case`:** Adiciona verificacao de cobertura completa pelo sintetizador, tornando o codigo mais robusto.
4. **Testbench com verificacao automatica:** A tarefa `testa_mux` com contagem de erros e relatorio PASS/FAIL e a abordagem correta para validacao de *designs* RTL em ambiente profissional.

A utilizacao do *testbench* para a aplicacao de vetores de teste nas 4 combinacoes de selecao validou a integridade do codigo RTL frente aos requisitos operacionais definidos na especificacao, confirmando a correta operacao do circuito e a conclusao bem-sucedida da atividade proposta.

Referencias Bibliograficas

- [1] IEEE STANDARDS ASSOCIATION. *IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2012. New York: IEEE, 2013.

- [2] EDA PLAYGROUND. *EDA Playground – Online HDL Simulator*. Disponivel em: <https://www.edaplayground.com/>. Acesso em: 1 mai. 2026.
- [3] WILLIAMS, S. *Icarus Verilog – Open-Source Verilog Simulator*. v12.0, 2023. Disponivel em: <http://iverilog.icarus.com/>. Acesso em: 12 mai. 2026.
- [4] RAZAVI, Behzad. *Design of Analog CMOS Integrated Circuits*. 2. ed. McGraw-Hill Education, New York, 2016.
- [5] BROWN, S.; VRANESIC, Z. *Fundamentals of Digital Logic with Verilog Design*. 3. ed. McGraw-Hill Education, New York, 2014.